

```

1  #!/usr/bin/env python3 -B
2  # ezr.py: explainable multi-objective optimization
3  # (c) 2026 Tim Menzies, timm@ieee.org, MIT license
4  """
5  Options:
6  --seed=1      random number seed
7  --p=2         distance (1,2=Man,Euclid)
8  --learn.leaf=3  examples per leaf
9  --learn.budget=50  rows to evaluate
10 --learn.check=5  guesses to check
11 --learn.start=4  initial labels
12 --bayes.m=2      m-estimate for Naive Bayes
13 --bayes.k=1      k-estimate (Laplace) for NB
14 --few=128        max unlabelled rows
15 --stats.cliffs=0.195 Cliff's Delta threshold
16 --stats.conf=1.36 K-S test confidence
17 --stats.eps=0.35 margin of error multiplier
18 --show.show=30   tree display width
19 --show.decimals=2 decimal places for floats
20 --textmine.norm=0 CNB weight normalization
21 --textmine.yes=20 positive samples
22 --textmine.no=20  negative samples
23 --textmine.top=100 top TF-IDF features
24 --textmine.valid=20 repeats for stats testing
25 """
26 from __future__ import annotations
27 from time import perf_counter_ns as now
28 import os, re, random, sys, bisect, math
29 from pathlib import Path
30 from random import random as rand
31 from random import choices, choice, sample, shuffle
32 from math import log, log2, exp, sqrt, pi
33 from typing import Any, Iterable, Callable
34 from types import SimpleNamespace as S
35
36 # i:self j:iter n,m:int txt:str
37 # v:val w:weight x:indep y:dep
38
39 isa = isinstance
40
41 # T y p a s
42 #
43 #
44 #
45
46 type Qty = int|float
47 type Atom = str|bool|Qty
48 type Row = list[Atom]
49 type Rows = list[Row]
50 type Col = Num|Sym
51 type Cols = list[Col]
52 type Tree = Any
53 type Datas = list[Data]
54
55 #
56 # C o l u m n s
57 #
58
59 def Col(txt="", a=0):
60     """Num or Sym column based on name case."""
61     return (Num if txt[0].isupper() else Sym)(txt, a)
62
63 class Num:
64     """Summarizes a stream of numbers."""
65     def __init__(i, txt="", a=0):
66         i.txt, i.at, i.n = txt, a, 0
67         i.mu=i.m2=i.sd=0; i.heaven=txt[-1:]!="-"
68
69 class Sym:
70     """Summarizes a stream of symbols."""
71     def __init__(i, txt="", a=0):
72         i.txt, i.at, i.n, i.has = txt, a, 0, {}
73
74 def mid(col):
75     """Central tendency (mean or mode)."""
76     return col.mu if Num==type(col) else max(
77         col.has, key=col.has.get)
78
79 def spread(col):
80     """Variability (sd or entropy)."""
81     if Num==type(col): return col.sd
82     n = sum(col.has.values())
83     return -sum(v/n*log2(v/n) for v in col.has.values())
84
85 def norm(num, v):
86     """Normalize via logistic function."""
87     if v == "?: return v
88     z=max(-3,min(3,(v-num.mu)/(num.sd+1e-32)))
89     return 1/(1 + exp(-1.7*z))
90
91

```

```

91 #
92 # D a t a
93 #
94
95 class Data:
96     """Rows + summarized columns."""
97     def __init__(i, src=None):
98         src = iter(src or {})
99         i.rows,i._centroid = [],None
100         i.cols = Cols(next(src))
101         adds(src, i)
102
103 class Cols:
104     """Organize Num/Sym columns from headers."""
105     def __init__(i, names):
106         i.names = names
107         i.klass, i.xs, i.ys, i.all = None,[],[],[]
108         for j,txt in enumerate(names):
109             i.all.append(col := Col(txt,j))
110             if txt[-1] != "X":
111                 if txt[-1] == "I": i.klass = col
112                 role = i.ys if txt[-1] in "+!" else i.xs
113                 role.append(col)
114
115 def clone(data, rows=None):
116     """Clone structure, optionally add rows."""
117     return adds((rows or [], Data([data.cols.names]))
118
119 def sub(it, v):
120     """Remove value/row (add with w=-1)."""
121     return add(it, v, -1)
122
123 def add(it, v, w=1):
124     """Add value/row to Data.Cols.Num.Sym."""
125     if Data is type(it):
126         it._centroid=None; add(it.cols,v,w)
127         if w>0: it.rows.append(v)
128         else : it.rows.remove(v)
129     elif Cols is type(it):
130         [add(col, v[col.at], w) for col in it.all]
131     elif v != "?:
132         if Sym == type(it):
133             it.has[v] = w + it.has.get(v, 0)
134         elif w < 0 and it.n <= 2:
135             it.n=it.mu=it.m2=it.sd=0
136         else:
137             it.n += w
138             delta = v - it.mu
139             it.mu += w * delta / it.n
140             it.m2 += w * delta * (v - it.mu)
141             it.sd=(max(0,it.m2)/(it.n-1))*0.5 if it.n>1 else 0
142     return v
143
144 def mids(data):
145     """Centroid of all columns."""
146     data._centroid = data._centroid or [
147         mid(col) for col in data.cols.all]
148     return data._centroid
149
150 def adds(src, it=None):
151     """Add multiple items to target."""
152     it = it or Num()
153     [add(it, v) for v in (src or [])]
154     return it
155
156

```

```

156 #
157 # D i s t a n c e
158 #
159
160 def minkowski(items):
161     """Minkowski distance (param: the.p)."""
162     tot, n = 0, 1e-32
163     for item in items:
164         tot, n = tot + item**the.p, n + 1
165     return (tot/n) ** (1/the.p)
166
167 def disty(data, row):
168     """Distance to heaven on Y vars."""
169     return minkowski(
170         abs(norm(y,row[y.at])-y.heaven)
171         for y in data.cols.ys)
172
173 def distx(data, r1, r2):
174     """Distance between rows on X vars."""
175     return minkowski(
176         aha(x,r1[x.at],r2[x.at]) for x in data.cols.xs)
177
178 def aha(col, u, v):
179     """Distance between two values."""
180     if u == v == "?: return 1
181     if Sym == type(col): return u != v
182     u, v = norm(col, u), norm(col, v)
183     u = u if u!="?" else (0 if v>0.5 else 1)
184     v = v if v!="?" else (0 if u>0.5 else 1)
185     return abs(u - v)
186
187 def nearest(data, row, rows=None):
188     """Closest row on x-columns."""
189     return min((rows or data.rows,
190         key=lambda r2: distx(data, row, r2))
191
192 #
193 # L i k e
194 #
195 #
196
197 def like(col, v, prior):
198     """How much a column likes a value."""
199     if type(col) == Sym:
200         return (col.has.get(v,0) +
201             the.bayes.k*prior)/(col.n+the.bayes.k)
202     sd=col.sd+1e-32; z=2*sd*sd
203     return exp(-(v-col.mu)**2/z)/sqrt(pi*z)
204
205 def likes(data, row, n_rows, n_klasses):
206     """Log likelihood of row given data."""
207     prior = (len(data.rows)+the.bayes.m
208         ) / (n_rows+the.bayes.m*n_klasses)
209     ls = [like(col,v,prior) for col in data.cols.xs
210         if (v:=row[col.at])!="?"]
211     return log(prior)+sum(log(v) for v in ls if v>0)
212
213 def classify(src, wait=10):
214     """Test then train: classify before update."""
215     src = iter(src)
216     h,cf,all = {}, None, Data([next(src)])
217     for n, row in enumerate(src):
218         want = row[all.cols.klass.at]
219         if n >= wait:
220             cf=dinc(want,max(h,key=lambda kl:likes(
221                 h[kl],row,len(all.rows),len(h))),cf)
222             if want not in h: h[want] = clone(all)
223             add(all, add(h[want], row))
224     return cf
225
226

```

```

226 # T i ̄ ̄ ̄ ̄ ̄ ( i ̄ ̄ ̄ ̄ ̄ ̄ )
227 #
228 #
229 #
230
231 class Tree:
232     """Decision tree node."""
233     def __init__(i, data, rows):
234         i.d = clone(data, rows)
235         i.ynum=adds((disty(data,row) for row in rows),Num())
236         i.col,i.cut = None,0
237         i.left=i.right=None
238
239 def treeCuts(col, rows):
240     """Possible split points for a column."""
241     vs=[row[col.at] for row in rows if row[col.at]!="?"]
242     if not vs: return []
243     return (set(vs) if Sym == type(col)
244             else [sorted(vs)[len(vs)//2]])
245
246 def treeSplit(data, col, cut, rows):
247     """Evaluate split on col at cut."""
248     l_rows,r_rows,l_num,r_num=[],[],Num(),Num()
249     for row in rows:
250         v = row[col.at]
251         go=v=="?" or (v==cut if Sym==type(col) else v<=cut)
252         (l_rows if go else r_rows).append(row)
253         add(l_num if go else r_num,disty(data,row))
254     s=l_num.n*spread(l_num)+r_num.n*spread(r_num)
255     return s,col,cut,l_rows,r_rows
256
257 def treeGrow(data, rows):
258     """Grow tree to minimize Y-variance."""
259     tree = Tree(data, rows)
260     if len(rows) >= 2 * the.learn.leaf:
261         splits = (treeSplit(data, col, cut, rows)
262                  for col in tree.d.cols.xs
263                  for cut in treeCuts(col, rows))
264         if valid := [s for s in splits
265                     if min(len(s[3]),len(s[4]))
266                        >= the.learn.leaf]:
267             _, tree.col, tree.cut, left, right = min(
268                 valid, key=lambda x: x[0])
269             tree.left = treeGrow(data, left)
270             tree.right = treeGrow(data, right)
271     return tree
272
273

```

```

273 # T i ̄ ̄ ̄ ̄ ̄ ( i ̄ ̄ ̄ ̄ ̄ ̄ )
274 #
275 #
276 #
277
278 def treeLeaf(tree, row):
279     """Find leaf node for row."""
280     if not tree.left: return tree
281     v = row[tree.col.at]
282     go = v!="?" and (v<=tree.cut
283                     if Num==type(tree.col) else v==tree.cut)
284     return treeLeaf(tree.left if go else tree.right,row)
285
286 def treeNodes(tree,lv1=0,col=None,op="",cut=None):
287     """Yield all nodes (depth-first)."""
288     yield tree, lv1, col, op, cut
289     if tree.col:
290         ops("<=", ">") if Num==type(
291             tree.col) else ("==", "!=")
292         kids=sorted([(tree.left,ops[0]),
293                     (tree.right,ops[1])],
294                     key=lambda z:mid(z[0].ynum))
295         for k, txt in kids:
296             if k: yield from treeNodes(k,lv1+1,
297                                         tree.col,txt,tree.cut)
298
299 def treeShow(tree):
300     """Print tree structure."""
301     for t1,lv1,col,op,cut in treeNodes(tree):
302         p=f"[col.txt] {op} {o(cut)}" if col else ""
303         if lv1 > 0: p = "| "*(lv1-1) + p
304         g=(col.txt:mid(col) for col in t1.d.cols.ys)
305         print(f"{p:<{the.show.show}}")
306         f",{o(mid(t1.ynum):>4)}"
307         f",{t1.ynum:n:3}), {o(g)}")
308
309 def treePlan(tree, here):
310     """Plans to improve from current leaf."""
311     eps = the.stats.eps * spread(tree.ynum)
312     for there,_,_,_,_ in treeNodes(tree):
313         if there.col is None and \
314            (dy:=mid(here.ynum)-mid(there.ynum))>eps:
315             diff=[f"[col.txt]={o(mid(col))}"
316                  for col,h in zip(there.d.cols.xs,here.d.cols.xs)
317                  if mid(col) != mid(h)]
318             if diff:
319                 yield dy, mid(there.ynum), diff
320
321

```

```

321 #
322 #
323 #
324 #
325 def acquireWithBayes(data, best, rest, row):
326     """Score: rest - best likelihood."""
327     n = len(best.rows)+len(rest.rows)
328     return likes(rest,row,n,2)-likes(best,row,n,2)
329
330 def acquireWithCentroid(data, best, rest, row):
331     """Score: dist(best) - dist(rest)."""
332     return (distx(data,row,mids(best)) -
333            distx(data,row,mids(rest)))
334
335 def warm_start(data, rows, label):
336     """Init lab/best/rest from start rows."""
337     lab = clone(data, rows[:the.learn.start])
338     lab.rows.sort(
339         key=lambda row:disty(lab,label(data,row)))
340     n = int(sqrt(len(lab.rows)))
341     return (lab,clone(data,lab.rows[:n]),
342            clone(data,lab.rows[n:]),rows[the.learn.start:])
343
344 def rebalance(best, rest, lab):
345     """Cap best at sqrt(|lab|); evict worst."""
346     if len(best.rows) > sqrt(len(lab.rows)):
347         best.rows.sort(key=lambda r:disty(lab,r))
348         add(rest.cols,sub(best.cols, best.rows[-1]))
349         rest.rows.append(best.rows.pop())
350
351 def acquire(data, score=acquireWithCentroid,
352            label=lambda _,row:row):
353     """Active learning. Returns labeled Data."""
354     rows = data.rows[:]
355     shuffle(rows)
356     lab,best,rest,unlab = \
357         warm_start(data, rows[:the.few], label)
358     fn = lambda row: score(lab,best,rest,row)
359     for _ in range(the.learn.budget):
360         if not unlab: break
361         pick, *unlab = sorted(unlab, key=fn)
362         add(lab, add(best, label(data, pick)))
363         rebalance(best, rest, lab)
364         lab.rows.sort(key=lambda r:disty(lab,r))
365     return lab
366
367

```

```

367 #
368 #
369 #
370 #
371
372 def picks(data, row, n=1):
373     """Mutate n random x-columns."""
374     s = row[:]
375     for col in sample(data.cols.xs,
376                       min(n, len(data.cols.xs))):
377         s[col.at] = pick(col, s[col.at])
378     return s
379
380 def oneplus1(data, mutate, accept, oracle,
381             budget=1000, restart=0):
382     """(1+1) search: mutate, score, accept."""
383     h, best, best_e = 0, None, 1E32
384     s, e, imp = choice(data.rows[:], 1E32, 0)
385     while h < budget:
386         for sn in mutate(s):
387             h += 1
388             en = oracle(sn)
389             if accept(e, en, h, budget):
390                 s, e = sn, en
391             if en < best_e:
392                 best, best_e, imp = sn[:], en, h
393             yield h, best_e, best
394             if restart and h - imp > restart:
395                 s = choice(data.rows[:])
396                 e, imp = 1E32, h
397             break
398
399 def oracleNearest(data, row):
400     """Score: copy y-vals from nearest."""
401     near = nearest(data, row)
402     for col in data.cols.ys:
403         row[col.at] = near[col.at]
404     return disty(data, row)
405
406 #
407 #
408 #
409
410 def same(xs, ys, eps):
411     """Are two lists statistically same?"""
412     xs, ys = sorted(xs), sorted(ys)
413     n, m = len(xs), len(ys)
414     if abs(xs[n//2]-ys[m//2]) <= eps: return True
415     gt = sum(bisect.bisect_left(ys,a) for a in xs)
416     lt=sum(m-bisect.bisect_right(ys,a) for a in xs)
417     if abs(gt-lt)/(n*m) > the.stats.cliffs:
418         return False
419     ks=lambda v:abs(bisect.bisect_right(xs,v)/n
420                    -bisect.bisect_right(ys,v)/m)
421     return max(max(map(ks,xs)),max(map(ks,ys))
422                ) <= the.stats.conf * ((n+m)/(n*m))*0.5
423
424 def bestRanks(d):
425     """Group treatments tied for best."""
426     items = sorted(d.items(), key=lambda kv:
427                   sorted(kv[1])[len(kv[1])/2])
428     k0, lst0 = items[0]
429     best = {k0: adds(lst0, Num(k0))}
430     for k, lst in items[1:]:
431         if same(lst0,lst,spread(best[k0])*the.stats.eps):
432             best[k] = adds(lst, Num(k))
433         else: break
434     return best
435
436

```

```

436 #
437 #
438 #
439
440 def dinc(k1, k2, b4=None):
441     """Increment nested dict counter."""
442     b4=b4 or {}; b4[k1]=b4.get(k1) or {}
443     b4[k1][k2] = b4[k1].get(k2, 0) + 1
444     return b4
445
446 def o(x):
447     """Recursive format. Sorts dicts."""
448     if isa(x, float):
449         return f"{x:.{the.show.decimals}f}"
450     if isa(x, dict):
451         return "{" + ",".join(f"{k}={o(v)}"
452                               for k,v in sorted(x.items()))+"}"
453     if isa(x, list):
454         return "[" + ",".join(map(o,x))+"]"
455     if isa(x, S): return "S" + o(x.__dict__)
456     if hasattr(x, "__dict__"):
457         return x.__class__.__name__+o(x.__dict__)
458     return str(x)
459
460 def table(lst, w=10):
461     """Print list of dicts as aligned table."""
462     if not lst: return
463     ds=[x if type(x) is dict else x.__dict__ for x in lst]
464     ks = list(ds[0].keys())
465     print("".join(f"{str(k):>{w}}" for k in ks))
466     print("-" * (len(ks) * w))
467     for d in ds:
468         print("".join(
469             f"{str(d.get(k,'')):>{w}}" for k in ks))
470
471 def thing(txt):
472     """Coerce string to number or bool."""
473     txt = txt.strip()
474     b=lambda s:{"true":1,"false":0}.get(s.lower(),s)
475     for f in [int, float, b]:
476         try: return f(txt)
477     except ValueError: pass
478
479 def nest(t, k, v):
480     """Set value in nested namespace."""
481     for x in (ks := k.split("."))[:-1]:
482         t = t.__dict__.setdefault(x, S())
483     setattr(t, ks[-1], v)
484
485 def csv(f, clean=lambda txt:
486        txt.partition("#")[0].split(",")):
487     """Yield typed rows from a CSV file."""
488     with open(f, encoding="utf-8") as file:
489         for txt in file:
490             row = clean(txt)
491             if any(x.strip() for x in row):
492                 yield [thing(x) for x in row]
493
494 def pick(it, v=None):
495     """Sample from distribution."""
496     if Sym == type(it): return pick(it.has)
497     if Num == type(it):
498         tmp=v if v is not None and v!="?" else it.mu
499         lo,hi=it.mu-3*it.sd,it.mu+3*it.sd
500         new=tmp+it.sd*2*(rand()+rand()+rand()-1.5)
501         return lo + (new-lo) % (hi-lo + 1e-32)
502     if dict == type(it):
503         n = sum(it.values()) * rand()
504         for k, v in it.items():
505             if (n := n - v) <= 0: break
506         return k
507
508

```

```

508 #
509 #
510 #
511 #
512
513 the = S()
514 for k,v in re.findall(r"([w.]+)=([S+])",__doc__):
515     nest(the, k, thing(v))

```